Informe Práctica 5B

Diseño de casos de prueba para ConjuntoOrdenado

Asignatura: Ingeniería del Software II
Práctica: 5B - Prueba de estructuras de datos
Alumno: Rodrigo Medrano Alonso
Curso: 2025/2026

1. Objetivo

El objetivo de esta práctica es diseñar y ejecutar pruebas unitarias sobre la clase ConjuntoOrdenado, que implementa la interfaz IConjuntoOrdenado.

Las pruebas se han diseñado aplicando técnicas de **caja negra**, concretamente **partición equivalente** y **análisis de valores límite**. Además, después de corregir los errores encontrados, se ha comprobado la cobertura mediante **JaCoCo** para validar las pruebas de caja blanca.

2. Métodos probados

La interfaz IConjuntoOrdenado define los siguientes métodos:

Método	**Descripción**
add(E elemento)	Añade un elemento en la posición que le corresponde según el orden natural.
get(int index)	Devuelve el elemento situado en la posición indicada.
remove(int index)	Elimina y devuelve el elemento situado en la posición indicada.
size()	Devuelve el número de elementos del conjunto.
clear()	Elimina todos los elementos del conjunto.

3. Diseño de casos de prueba de caja negra

Los casos de prueba se diseñaron a partir de la especificación de la interfaz, sin tener en cuenta inicialmente la implementación interna de la clase.

3.1. Partición equivalente

Se identificaron clases de equivalencia válidas e inválidas para cada operación.

Método add

Caso	Resultado esperado
Añadir un elemento en un conjunto vacío	El elemento se añade y el método devuelve true.
Añadir un elemento menor que los existentes	El elemento queda colocado al inicio.
Añadir un elemento intermedio	El elemento queda colocado en la posición correcta.
Añadir un elemento mayor que los existentes	El elemento queda colocado al final.
Añadir un elemento duplicado	El método devuelve false y el conjunto no cambia.
Añadir null	Se lanza NullPointerException.

Método get

Caso	Resultado esperado
Índice válido	Devuelve el elemento correspondiente.
Índice negativo	Lanza IndexOutOfBoundsException.
Índice igual al tamaño del conjunto	Lanza IndexOutOfBoundsException.
Conjunto vacío	Lanza IndexOutOfBoundsException.

Método remove

Caso	Resultado esperado
Eliminar el primer elemento	Devuelve y elimina el primer elemento.
Eliminar un elemento intermedio	Devuelve y elimina el elemento correcto.
Eliminar el último elemento	Devuelve y elimina el último elemento.
Índice negativo	Lanza IndexOutOfBoundsException.
Índice igual al tamaño del conjunto	Lanza IndexOutOfBoundsException.
Conjunto vacío	Lanza IndexOutOfBoundsException.

Métodos size y clear

Caso	Resultado esperado
Conjunto recién creado	size() devuelve 0.
Después de añadir elementos	size() devuelve el número correcto de elementos.
Después de eliminar elementos	size() se actualiza correctamente.
Después de ejecutar clear()	El conjunto queda vacío.
Ejecutar clear() sobre un conjunto vacío	No se produce error.

4. Análisis de valores límite

El análisis de valores límite se aplicó principalmente a los métodos que reciben índices: get y remove.

Para un conjunto de tamaño n, se probaron los siguientes valores:

Valor	Tipo	Resultado esperado
-1	Inválido	Excepción
0	Válido si el conjunto no está vacío	Primer elemento
Posición intermedia	Válida	Elemento correspondiente
n - 1	Válido	Último elemento
n	Inválido	Excepción

También se probaron los límites de inserción en add, comprobando la inserción al inicio, en medio y al final del conjunto.

5. Casos de prueba implementados

Se implementaron 16 pruebas unitarias con JUnit 5.

Los casos diseñados cubren:

- Creación de un conjunto vacío.
- Inserción de elementos.
- Mantenimiento del orden natural.

- Rechazo de elementos duplicados.
- Rechazo de elementos nulos.
- Acceso con índices válidos e inválidos.
- Eliminación en primera, media y última posición.
- Eliminación con índices inválidos.
- Comprobación del tamaño.
- Vaciado del conjunto con clear.
- Reutilización del conjunto después de vaciarlo.

6. Ejecución inicial sobre la versión base

Primero se ejecutaron las pruebas sobre la versión original de ConjuntoOrdenado, sin corregirla previamente.

El resultado fue:

Tests run: 16, Failures: 3, Errors: 0, Skipped: 0
BUILD FAILURE

Los tests fallidos fueron:

addMantieneOrdenNaturalAscendenteEnInicioMedioYFinal
addDuplicadoDevuelveFalseYNoModificaElConjunto
removeEnPrimeraMediaYUltimaPosicion

Estos fallos permitieron detectar defectos en el método add.

El primer error indica que los elementos no se estaban almacenando en orden natural ascendente. El segundo error indica que se permitían elementos duplicados, aunque la estructura debe comportarse como un conjunto. El tercer error aparece en una prueba de remove, pero está causado por el orden incorrecto producido previamente por add.

7. Corrección realizada

El método defectuoso era add.

La versión original avanzaba por la lista con una comparación incorrecta:

elemento.compareTo(lista.get(indice)) < 0

Esto provocaba una ordenación descendente.

Además, el método no comprobaba si el elemento ya existía, por lo que siempre insertaba el elemento y siempre devolvía true.

La corrección consistió en:

1. Cambiar la comparación para mantener orden ascendente.
2. Comprobar si el elemento ya existe antes de insertarlo.
3. Devolver false si el elemento está duplicado.

8. Ejecución final sobre la versión corregida

Después de corregir el método add, se volvieron a ejecutar las pruebas unitarias.

El resultado fue:

Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

Esto confirma que la implementación corregida supera todos los casos de prueba diseñados.

9. Cobertura de caja blanca

Finalmente, se comprobó la cobertura usando JaCoCo.

El informe mostró los siguientes resultados:

Métrica	**Resultado**
Cobertura de instrucciones	100 %
Cobertura de ramas	100 %
Líneas cubiertas	17/17
Métodos cubiertos	6/6
Clases cubiertas	1/1

Por tanto, se consiguió:

Cobertura de sentencias: 100 %
Cobertura de decisión/condición: 100 %

Al alcanzarse una cobertura completa, no fue necesario añadir nuevos casos de prueba.

10. Conclusión

Se diseñó una batería de 16 pruebas unitarias aplicando partición equivalente y análisis de valores límite.

La ejecución inicial sobre la versión base permitió detectar errores en el método add, concretamente en el mantenimiento del orden natural y en la gestión de elementos duplicados.

Tras corregir dichos defectos, todas las pruebas pasaron correctamente. Además, el análisis con JaCoCo confirmó una cobertura del 100 % tanto en sentencias como en ramas, por lo que la batería de pruebas se considera suficiente para validar la clase ConjuntoOrdenado.